

ECE 474A/574A: Computer-Aided Logic Design

Lab Assignment #3 - MLP Classifier

1 Preliminaries

- Read Chapter 7 of “Parallel Programming for FPGAs”.
- Review lecture slides of Matrix Multiplication design in “Lec5-Dense-Matrix-Multiplication”.
- Download the project files from D2L: mlp.h, mlp.cpp, mlp_test.cpp, and mlp.ipynb

2 Introduction

The goal of this lab is to analyze the implementation of dense matrix multiplication in **Multilayer Perceptron (MLP)** classifier and evaluate its impact on performance, resource utilization, and computational efficiency. Students will use Vitis HLS to explore different design trade-offs, optimizing the computation and memory bandwidth bottleneck of dense matrix multiplication while managing hardware constraints.

A **classifier** is a type of machine learning model that is trained to categorize input data into predefined classes or categories. In this assignment, we use an MLP classifier to identify handwritten digits from the **MNIST (Modified National Institute of Standards and Technology)** dataset. Each input image corresponds to a digit (0–9), and the classifier predicts the correct class label.

Multilayer Perceptrons (MLPs) are a class of feedforward neural networks composed of layers of interconnected neurons as shown in Figure 1. An MLP typically includes:

- **Input layer** — receives the raw data (e.g., pixel values). The MNIST dataset consists of grayscale images of size 28×28 pixels, flattened into 784-dimensional vectors.
- **Hidden layers** — perform feature extraction using linear and nonlinear transformations.
- **Output layer** — generates the final classification result, typically using a softmax function.

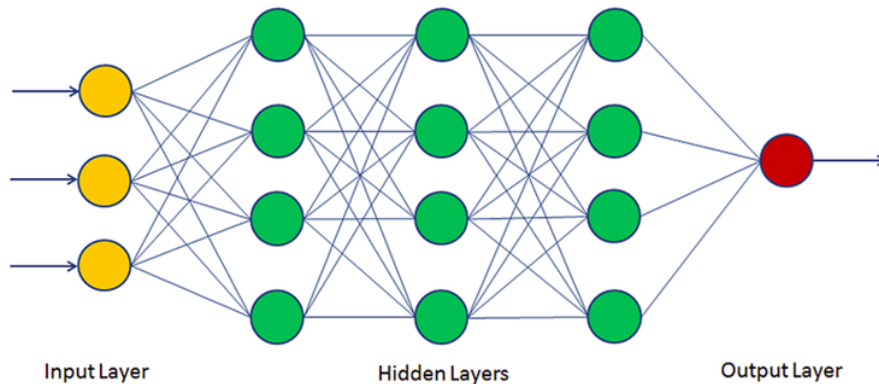


Figure 1: Structure of a simple Multilayer Perceptron with 3 Hidden Layers.

In this assignment, we use a three-layer MLP trained on MNIST. The architecture is as follows:

- **Input:** 128×784 image matrix
- **Hidden Layer 1:** 1024 neurons

- **Hidden Layer 2:** 512 neurons
- **Output Layer:** 10 neurons (corresponding to 10 digit classes)

Each layer performs a linear transformation followed by a non-linear activation function. The dominant computation in each layer is the matrix-vector multiplication Wx . The linear transformation is defined by

$$y = Wx + b \quad (1)$$

where x is the input vector, W is the weight matrix, b is the bias vector, and y is the output of the layer.

To maximize hardware efficiency, we compute on a **fixed-size batch (e.g., 128) of inputs**. Instead of processing one image at a time, we process multiple inputs simultaneously. This transforms the operation into a matrix-matrix multiplication, where the input batch is represented as a matrix X and the output is:

$$Y = WX + b \quad (2)$$

Each **fully connected (FC)** layer performs a matrix-matrix multiplication between a weight matrix and the input matrix:

- Hidden Layer 1: $W_1[1024 \times 784]$ multiplies with *input* $[784 \times 128]$, results in $y_1[1024 \times 128]$
- Hidden Layer 2: $W_2[512 \times 1024]$ multiplies with $y_1[1024 \times 128]$, results in $y_2[512 \times 128]$
- Output Layer: $W_3[10 \times 512]$ multiplies with $y_2[512 \times 128]$, results in *output* $[10 \times 128]$

Thus, this MLP involves three dense matrix-matrix multiplications, followed by element-wise non-linear activations (ReLU for hidden layers, softmax for output).

Matrix multiplication is a fundamental operation in numerical computing, combining two matrices to produce a third. It serves as a core computation in various domains, including scientific computing, machine learning, and signal processing. Traditional matrix multiplication follows a well-defined linear algebraic process, where the product of an $n \times m$ matrix and an $m \times p$ matrix results in an $n \times p$ matrix.

Block matrix multiplication partitions input matrices into smaller sub-matrices (blocks) and processes them independently, computing partial results before assembling the final output. This method is particularly beneficial for large-scale matrix operations as it improves memory access patterns and enables efficient parallel execution. Block-based techniques are widely used in high-performance computing applications, including graphics transformations, statistical modeling, and graph-based computations. Optimizing matrix multiplication through block processing is a well-studied problem, with various algorithms developed to enhance speed and reduce memory overhead.

Given 4×4 matrices A and B and with a block size of 2×2 , 4×4 matrix C is given by:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}$$

3 Goals

Our objective is to implement and accelerate the matrix multiplication of FC layers at the system level. The assignment has three main sections:

- **Software Implementation on PYNQ Processing System (PS):** You will implement the baseline matrix multiplication algorithm in the baseline Python code and deploy it on the PYNQ PS to establish a software reference for comparison.
- **Hardware Implementation in HLS:** You will modify the provided baseline code to design block matrix multiplication with various performance enhancements, including pipelining, loop unrolling, array partitioning, wide bus data types with transposing matrix A, and optimizing dataflow to improve memory access patterns. Implement the design on the Programmable Logic (PL) using Vitis HLS and analyze its resource utilization and latency. Explore the block size to perform design space exploration, explore trade-offs between latency, resource usage, and throughput, and maximize GOPS.

- **PS-PL System Integration and Performance Analysis:** You will integrate the software and hardware implementations to enable efficient PS-PL communication. Evaluate speedup, resource usage, and overall efficiency compared to the baseline implementation.

4 Performance Metrics

The design performance is evaluated based on the following metrics:

- **Resource Utilization:** The hardware resources used by the design, including LUTs, FFs, DSPs, BRAMs, as reported in the Place and Route report from the implementation stage.
- **Latency:** The overall time required to complete the operation execution, where the total number of clock cycles is reported in the Synthesis report from the C Synthesis step. You should use the “Estimated Clock Period” from the HLS report instead of the specified clock period.

$$\text{Latency} = \text{Clock Period (s)} \times \text{Total Clock Cycles}$$

- **Giga Operation Per Second (GOPS):** The performance metric that quantifies the computational throughput, measuring the number of billion operations per second.

$$\text{GOPS} = \frac{2 * (N * M * P)}{\text{Total Latency (ns)}}$$

5 Software Implementation on PYNQ Processing System (PS): (10 points)

Let’s first review matrix multiplication and its computational complexity. The conventional method has a time complexity of $O(N^3)$, meaning that for each of the N^2 elements in the output matrix, we perform N MAC (multiply and accumulate) operations. Beyond computation, memory access is also a major factor—each of the N^2 elements is accessed N times. This applies regardless of whether the computation is done on a CPU or FPGA. As N grows, both computational load and memory access demands become enormous.

In total, computing the output matrix requires N^3 MAC operations, and approximately $2N^3$ memory accesses. On a CPU, libraries like NumPy provide optimized implementations of matrix multiplication, making it as simple as a function call. Extensive research has led to faster algorithms that reduce the time complexity below $O(N^3)$, e.g. $O(N^{2.371339})$ for Alman’s algorithm [ADW+25].

The Part I of lab is to implement and evaluate matrix multiplication in Python. The code should be added to the corresponding section (5) of the provided Jupyter Notebook (“mlp.ipynb”).

1. Implement a function that takes the weight matrix $W[N \times M]$, input matrix $X[M \times P]$, bias vector $b[N]$, and the sizes N , M , P as inputs. The function should return the output matrix $Y[N \times P]$ computed as in Equation (2).
2. Assume square matrices of the same size, i.e., $N = M = P$. Use NumPy to create two square matrices and a bias vector filled with random nonzero values.
3. Once the correctness of your function has been verified, measure the elapsed time before and after performing the matrix multiplication.
4. Compare computation times for square matrices of sizes 128, 256, 512, 1024, 2048, and 4096.

Question 1: Report the performance of your design for these input sizes. How does increasing matrix size impact the computation time?

6 Hardware Implementation in HLS: (70 points)

In Part II, you should generate multiple designs by applying various optimizations **incrementally** step-by-step, creating distinct designs for each. For each subsection, you need to answer some questions in your report. Optimized designs must always maintain functional correctness by comparing the generated outputs with the golden reference outputs. In your report, explain your answer to the questions in detail. Note that a well-reasoned analysis that demonstrates a deep understanding of the characteristics of each optimization is more credible than simply reporting numerical results.

Note: Keep in mind to report the specified performance metrics to evaluate the design of each optimization, as summarized in Table 1.

6.1 Baseline Block Matrix Multiplication

In this section, you will implement a baseline version of block matrix multiplication without any optimizations. Your design should use **int** as the data type, accept the matrix sizes as input, and **set the block sizes to a fixed value of 16**.

Question 2: When computing each block output, you have three possible loop orderings. Implement and analyze each version. Verify your design using a testbench and report its correctness. Compare performance metrics across different loop orderings and concisely analyze your findings.

6.2 Optimized Block Matrix Multiplication

Let's use loop ordering $k \rightarrow i \rightarrow j$ (in Figure 2) for blocked matrix multiplication when applying various optimizations and exploring design spaces.

6.2.1 Pipeline/Unroll

In this section, you will apply pipelining to the inner loops responsible for computing each output block of your design. Applying loop pipelining results in the unrolling of inner loops, requiring simultaneous memory reads and writes.

Question 3: Choose the loop that provides the best trade-off between throughput and resource consumption. **Justify why this choice is optimal.** To enable multiple concurrent accesses, apply array partitioning to the matrices. However, **avoid unnecessary partitioning**—only partition dimensions that impact output performance to optimize resource utilization. Analyze and report key performance metrics accordingly.

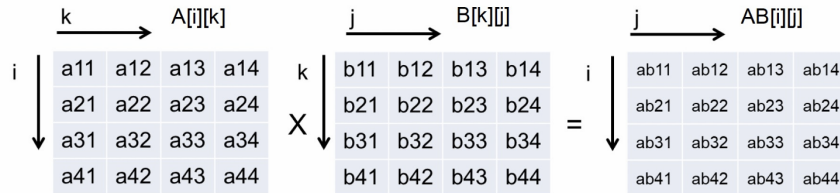


Figure 2: Block Matrix Multiplication

6.2.2 Wide Bus Data Type with Transposed Matrix A

While pipelining enhances computational performance, insufficient memory bandwidth can limit efficiency by failing to supply data to all MAC units.

Moreover, matrix A is accessed column-wise, resulting in uncoalesced memory accesses and inefficient use of available memory bandwidth. To address this, you can transpose matrix A so that access patterns become row-wise and more memory-friendly. To maximize hardware efficiency, combine the use of a **wide bus data type** with the **transposition of matrix A** .

Hint: Instead of implementing a separate hardware kernel to transpose the matrix A , perform the transposition in the previous layer of the MLP **while writing to the output matrix**. This approach eliminates additional overhead and simplifies the top-level hardware design.

Question 4: Modify your function arguments to use a wide data type, specifically a `hls::vector`. Update data transfer routines to ensure compatibility with the new vectorized data type with the transposed version of matrix A . Report key performance metrics and compare them with those of the previous implementation, discussing the impact of this combined optimization.

6.2.3 Dataflow

To further optimize the performance of block matrix multiplication, the **dataflow** optimization should be applied to enable better overlap between computation and memory access. This technique allows different stages of the computation pipeline—such as loading data blocks, performing matrix multiplication, and writing results—to execute concurrently, improving throughput and overall efficiency.

Question 5: Apply the **DATAFLOW** directive to your design, making the necessary modifications to your code (e.g., using `hls::stream` and separating tasks into parallel execution functions). Report the performance improvement compared to the previous implementation without dataflow, and explain which stages benefit the most from the overlapping.

7 PS-PL System Integration: (20 points)

In this part of the assignment, you will integrate the software and hardware components to create a fully functional system. The PYNQ processing system (CPU) will generate the input data and the corresponding golden reference values, as implemented in Section 5. You are expected to complete the corresponding section (“Deploying MLP on PYNQ”) of the provided template Jupyter Notebook file (“`mlp.ipynb`”) to feed inputs to your IP, instantiate your IP design, and read the output matrix to be passed to the next component. Following the tutorial on **Deployment on FPGA**, you will configure the FPGA with the hardware overlay, write the input data to memory, and evaluate the output. This process reflects a typical workflow in real-world FPGA applications. Note the FPGA board demo is required.

Question 6: Use the **INTERFACE** directive to explicitly assign design ports to the Master AXI interface, ensuring proper connectivity to AXI ports for memory operations. Follow the steps outlined in the FPGA deployment tutorial. You may need to modify or convert the generated data to match the expected input format of your hardware design. Verify the functionality by comparing the hardware output with the golden reference value produced by the software. Finally, measure and report the execution time of your hardware design.

8 Reporting

Please format your report in a 2-page single-column, single-spaced layout with an 10pt font size. The report should clearly outline its purpose, provide an overview of both baseline and optimized designs, and explain the experiments by integrating responses to key questions. Use tables or plots to highlight significant findings and major results. Use screenshots only for illustrating the most significant insights. Additionally, you should include Table 1 in your report summarizing the evaluation results using the metrics mentioned in Section 4. This will help you visualize the trends and provide evidence to support your answers to the corresponding questions. Ensure all figures and tables have captions with proper explanations.

9 Deliverables

Remember **DO NOT SUBMIT THE ENTIRE HLS FOLDER**. You can find the synthesis and implementation reports (`mlp_csynth.rpt` and `mlp_export.rpt`) in “<your HLS component folder>/hls/syn/report” and “<your HLS component folder>/hls/impl/report/verilog”, respectively.

Table 1: Summary of key performance metrics

Design	Resource Utilization				Latency (Cycles)	Estimated Clk Period (ns)	Latency (ns)	GOPS
	LUT	FF	DSP	BRAM				
Baseline (ikj)								
Baseline (kji)								
Baseline (kij)								
+ Pipeline/Unroll								
+ WideBus/Transposed								
+ Dataflow								

This assignment is a group lab with two/three members. **Each group only need to submit a single report and implementation files.**

Upload a compressed version of the assignment folder named `Member1FirstName-Member1LastName_Member2FirstName-Member2LastName_Lab3` (e.g., `Hanieh-TotonchiAsl.Ebrahim-Nouri_Lab3`), including the following files:

- **Report.pdf**
- Folder **mlp_baseline**: `mlp.h`, `mlp.cpp`, `mlp_csynth.rpt`, `mlp_export.rpt`
- Folder **mlp_pipeline_unroll**: `mlp.h`, `mlp.cpp`, `mlp_csynth.rpt`, `mlp_export.rpt`
- Folder **mlp_wide_bus_transposed**: `mlp.h`, `mlp.cpp`, `mlp_csynth.rpt`, `mlp_export.rpt`
- Folder **mlp_dataflow**: `mlp.h`, `mlp.cpp`, `mlp_csynth.rpt`, `mlp_export.rpt`
- Folder **Demo**: `mlp.bit`, `mlp.hwh`, `mlp.ipynb`
- Optimizations should be applied **incrementally**, with each optimized design stored in a separate folder. Avoid including multiple optimizations in the same file by commenting them out. You may need to experiment with different parameters for each optimization to address the given questions. However, only include the files of the design that achieve the best performance.

References

- [ADW⁺25] J. Alman, R. Duan, V. Vassilevska Williams, Y. Xu, Z. Xu, and R. Zhou. More asymmetry yields faster matrix multiplication. *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2005–2039, 2025. [Online]. Available: <https://epubs.siam.org/doi/abs/10.1137/1.9781611978322.63>.